Mutabilidad e inmutabilidad de los datos: del const de C a la inmutabilidad en lenguajes modernos

Estudiante: Johan Chamba
Carrera de Computación
Primer Ciclo
https://orcid.org/0009-0004-8274-9695

Resumen

En este ensayo se analiza el concepto de mutabilidad e inmutabilidad de los datos. Este es un tema transversal en programación. Desde el const de C hasta los mecanismos de lenguajes modernos (Java, C#, JavaScript, Rust). Se realiza una revisión bibliográfica narrativa de estos mecanismos. Se comparan las garantías que ofrecen. Se contrasta con la evidencia de las dificultades que tienen los estudiantes de programación. Los resultados muestran que los mecanismos habituales (const, final, readonly) restringen principalmente la reasignación del identificador o el acceso por el identificador. Raramente ofrecen inmutabilidad profunda. Adicionalmente, se observa que las malas concepciones sobre datos mutables son frecuentes en la literatura. Estas no se corrigen espontáneamente con la experiencia. Se concluye que es importante enseñar la inmutabilidad como un espectro de garantías y no como una palabra clave.

Palabras clave— *mutabilidad; inmutabilidad; const; educación en programación; lenguajes de programación.*

I. INTRODUCCIÓN

En programación imperativa los programas tienen un estado que evoluciona a lo largo de la ejecución. Los procedimientos pueden tener efectos secundarios además del valor de retorno. La idea de inmutabilidad pasa por datos que no cambian tras su creación. El primer contacto con esta idea para el estudiante suele pasar por la constante (const en C, final en Java, readonly en C#). Las palabras clave no son equivalentes. Ofrecen diferentes garantías.

Hay un problema doble. Por un lado los mecanismos varían en lo que restringen y en la profundidad. Por otro lado, en la literatura de educación se observa que las malas concepciones sobre variables mutables y datos compuestos mutables son frecuentes [1]. Incluso en estudiantes de cursos avanzados [2]. Desde el primer año hay muchos estudiantes con modelos mentales no viables de la asignación por valor y por referencia [3]. El objetivo de este ensayo es analizar los mecanismos de inmutabilidad en varios lenguajes. Analizar sus implicaciones para la enseñanza de la programación. Las contribuciones son: (i) un criterio para comparar garantías de inmutabilidad; (ii) una tabla comparativa de mecanismos; (iii) una síntesis de la evidencia educativa. La estructura del documento es la siguiente. En la Sección II se presentan los resultados. En la Sección III se discuten los resultados. Finalmente en la Sección IV se presentan las conclusiones.

II. RESULTADOS

A. Comparación de mecanismos

Para la comparación se usa el siguiente criterio. La garantía es superficial si solo cubre la referencia. Es profunda si cubre todos los objetos accesibles de manera transitiva. En la Tabla I se resume la situación de los mecanismos analizados. En C, Java, C#, JavaScript el mecanismo restringe un identificador o campo. No restringe el contenido del valor. Solo en Rust se extiende a valor poseído (con algunas excepciones como mutabilidad interior).

TABLA I
COMPARACIÓN DE MECANISMOS DE INMUTABILIDAD

Lenguaje	Mecanismo	Qué restringe	Profundidad
C	const	Modificar el dato por ese acceso	Superficial; elusible con conversión
Java	final	Reasignar variable o campo	Superficial (referencia)
C#	readonly	Asignar el campo fuera de constructor	Superficial (referencia)
JavaScript	const	Reasignar el enlace	Superficial (referencia)
Rust	let (sin mut)	Mutar a través del enlace	Mayor; admite mutabilidad interior

Fuente: elaboración propia.

B. Un espectro de garantías

Los mecanismos pueden ordenarse conceptualmente desde datos sin restricciones hasta inmutabilidad profunda (Tabla II). Es una simplificación. Por ejemplo restringir enlace y restringir camino de acceso son mecanismos diferentes. Pero en ambos casos la garantía es superficial.

TABLA II
ESPECTRO CONCEPTUAL DE GARANTÍAS DE INMUTABILIDAD

Nivel	Garantía	Ejemplos
1	Mutable sin restricciones	int x; objeto sin calificadores
2	Restricción de enlace o de acceso	const (C, JS), final (Java), readonly (C#)
3	Inmutabilidad superficial	Object.freeze (JS), tuplas (Python)
4	Inmutabilidad profunda	Datos persistentes (OCaml); Rust (valores propios)

Fuente: elaboración propia. A mayor nivel mayor garantía.

C. Evidencia educativa

Los estudios revisados muestran cuatro hallazgos. Primero que las malas concepciones sobre mutación son frecuentes [1]. Segundo que persisten en estudiantes de años superiores. No hay transferencia natural entre lenguajes [2]. Tercero que desde el primer año hay muchos estudiantes con modelos mentales no viables de la asignación por valor y por

referencia. Esto se asocia con peor desempeño [3]. Cuarto que incluso con instrucciones los estudiantes que usan final en Java no son capaces de expresar correctamente la inmutabilidad [4]. Los profesionales demandan formas de expresar inmutabilidad que los lenguajes de uso común no ofrecían [5].

III. DISCUSIÓN

Los resultados sugieren que el hecho de que el primer contacto sea con la palabra const puede inducir a una expectativa que el lenguaje no cumple. Los estudiantes pueden pensar que el dato "no cambia". Sin embargo sólo se restringe un enlace o camino de acceso (ver Tabla I). Esto es coherente con que no haya transferencia automática entre lenguajes [2]. Los estudiantes que aprenden final en Java no son capaces de inferir por sí mismos las diferencias con const en C o JavaScript.

También respalda la idea de explicitar y verificar la inmutabilidad. No basta con una palabra clave. En el estudio de Glacier se muestra que en la práctica final es insuficiente (incluso con instrucciones) [4]. Una implicación didáctica es enseñar la inmutabilidad como un espectro (ver Tabla II). Es conveniente mostrar ejemplos de lo que no protege cada palabra clave. También es interesante trabajar el mismo programa en más de un lenguaje.

Este trabajo tiene algunas limitaciones. Es una revisión narrativa (con un número reducido de fuentes). No es una revisión sistemática. La evidencia educativa que se cita se refiere a malas concepciones sobre mutación en general. No sobre la comprensión de const, final o readonly. Esto se plantea como hipótesis.

IV. CONCLUSIONES

Los mecanismos (const, final, readonly) restringen principalmente la reasignación de un identificador o el acceso por el identificador. Raramente ofrecen inmutabilidad profunda. Sólo los lenguajes que parten de inmutabilidad (como Rust) extienden la restricción al valor propio. La literatura muestra que las malas concepciones sobre datos mutables son frecuentes. Es importante enseñar la inmutabilidad como un espectro de garantías y no como una palabra clave.

Como trabajo futuro se plantea replicar la revisión (como revisión sistemática). Diseñar un estudio con estudiantes de CS1 que permita evaluar de manera directa la comprensión de const, final y readonly entre lenguajes.

REFERENCIAS

[1] K.-C. Lu and S. Krishnamurthi, "Identifying and correcting programming language behavior misconceptions," Proc. ACM Program. Lang., vol. 8, no. OOPSLA1, Art. 106, pp. 334–361, Apr. 2024, doi: 10.1145/3649823.

[2] K. Fisler, S. Krishnamurthi, and P. Tunnell Wilson, "Assessing and teaching scope, mutation, and aliasing in upper-level undergraduates," in Proc. ACM Tech. Symp. Computer Science Education (SIGCSE), 2017.

[3] L. Ma, J. Ferguson, M. Roper, and M. Wood, "Investigating and improving the models of programming concepts held by novice programmers," Comput. Sci. Educ., vol. 21, no. 1, pp. 57–80, Mar. 2011, doi: 10.1080/08993408.2011.554722.

[4] M. Coblenz, W. Nelson, J. Aldrich, B. Myers, and J. Sunshine, "Glacier: Transitive class immutability for Java," in Proc. 39th Int. Conf. Software Engineering (ICSE), Buenos Aires, Argentina, 2017, pp. 496–506, doi: 10.1109/ICSE.2017.52.

[5] M. Coblenz, J. Sunshine, J. Aldrich, B. Myers, S. Weber, and F. Shull, "Exploring language support for immutability," in Proc. 38th Int. Conf. Software Engineering (ICSE), 2016.